

NGHIÊN CỨU KỸ THUẬT REVERSE ENGINEERING BINARY TRÊN THIẾT BỊ NHÚNG ARM

1. Tổng Quan Dự Án

1.1. Giới Thiệu

Reverse engineering (RE) binary là kỹ năng cốt lõi trong lĩnh vực bảo mật phần mềm nhưng — cho phép nhà nghiên cứu hiểu hoạt động của phần mềm mà không có mã nguồn gốc, phát hiện lỗ hổng ẩn, phân tích mã độc firmware, và đánh giá mức độ bảo mật của thiết bị IoT. Khác với RE trên môi trường x86 desktop, RE trên ARM embedded đòi hỏi hiểu biết sâu về kiến trúc ARM (Thumb/ARM mode, calling convention, exception model), cấu trúc ELF cho cross-compile, và môi trường thực thi đặc thù của hệ thống nhúng không có MMU đầy đủ.

Kiến trúc ARM chiếm hơn 95% thị trường thiết bị nhúng hiện đại: từ router gia đình (ARM Cortex-A7/A9), camera IP, smart TV, đến hệ thống ô tô và thiết bị y tế. Hầu hết firmware của các thiết bị này được compile thành ELF binary ARM stripped — không có symbol, không có debug info, đôi khi bị obfuscate — đòi hỏi kỹ năng RE chuyên sâu để phân tích.

Đề án này xây dựng một quy trình RE binary ARM hoàn chỉnh và có hệ thống, bao gồm: phân tích tĩnh với Ghidra và IDA Free, emulation động với QEMU, debugging từ xa với GDB multiarch, phân tích hành vi runtime với strace/ltrace, và ứng dụng RE để phát hiện lỗ hổng bảo mật trong firmware nhúng ARM thực tế.

1.2. Mục Tiêu Dự Án

Đề án hướng đến các mục tiêu nghiên cứu chuyên sâu sau:

- Nắm vững kiến trúc ARM (32-bit và 64-bit): instruction set, thanh ghi, calling convention AAPCS/AAPCS64, Thumb/Thumb-2 mode
- Xây dựng quy trình phân tích tĩnh binary ARM bằng Ghidra: import, analysis, decompile, scripting tự động
- Thành thạo IDA Pro Free / Radare2 như công cụ hỗ trợ cho RE ARM
- Triển khai môi trường QEMU emulation đầy đủ cho ARM user-mode và system-mode
- Thực hiện GDB remote debugging binary ARM đang chạy trong QEMU với pwndbg/peda
- Phân tích runtime behavior với strace, ltrace, valgrind ARM
- Ứng dụng RE để xác định và xác minh lỗ hổng: stack overflow, format string, logic flaw

- Phân tích ít nhất 2 firmware ARM thực tế (stripped binary): httpd, telnetd, custom daemon
- So sánh diff 2 phiên bản firmware (BinDiff) để xác định security patch
- Xây dựng Ghidra script tự động hóa quy trình RE cho firmware ARM

1.3. Phạm Vi Dự Án

Phạm vi bao gồm:

- Kiến trúc: ARM Cortex-A (ARMv7-A 32-bit và ARMv8-A 64-bit), cả ARM mode và Thumb/Thumb-2
- Target binary: ELF stripped, dynamically linked và statically linked, ARM little-endian
- Công cụ static: Ghidra 11, IDA Free 8, Radare2/Cutter, readelf, objdump, strings, checksec
- Công cụ dynamic: QEMU 8 (user-mode + system-mode), GDB multiarch + pwndbg, strace, ltrace
- Firmware target: Router ARM (TP-Link Archer), IP Camera ARM (firmware EoL)
- Ứng dụng: Tìm lỗ hổng qua RE — stack overflow, format string, command injection trong httpd
- Ghidra scripting: Java API và Python (Jython) để tự động hóa rename, phân tích

Phạm vi ngoài đồ án:

- Phát triển exploit hoàn chỉnh (chỉ dừng ở xác định và xác minh lỗ hổng qua RE)
- RE binary MIPS, x86 — tập trung hoàn toàn ARM
- Tấn công phần cứng (side-channel, fault injection)
- Phân tích mã độc (malware) — chỉ RE firmware hợp lệ của thiết bị EoL

2. Kiến Trúc Hệ Thống

2.1. Kiến Trúc ARM — Nền Tảng Lý Thuyết

Trước khi RE, cần nắm vững các đặc điểm kiến trúc ARM ảnh hưởng trực tiếp đến quá trình phân tích binary:

Đặc điểm	ARM 32-bit (ARMv7-A)	ARM 64-bit (ARMv8-A / AArch64)
Thanh ghi general	R0–R12, SP(R13), LR(R14), PC(R15)	X0–X30, SP, PC riêng biệt
Return address	Lưu trong LR (R14), không trên stack	Lưu trong LR (X30), không trên stack
Calling convention	AAPCS: args R0–R3, return R0	AAPCS64: args X0–X7, return X0
Instruction mode	ARM (32-bit) + Thumb (16-bit) + Thumb-2	Chỉ AArch64 (32-bit fixed)
Branch & link	BL label (lưu return vào LR)	BL label (lưu return vào X30)
Stack frame	PUSH {fp, lr} / POP {fp, pc}	STP x29, x30, [sp, #-N]!
Syscall	SWI #0 / SVC #0, số trong R7	SVC #0, số trong X8
Endianness	Thường Little-endian (LE)	Little-endian (LE)

Thumb Mode — Điểm Đặc Biệt Quan Trọng Khi RE

Thumb là instruction set 16-bit của ARM, được dùng rộng rãi trong firmware nhưng để tiết kiệm bộ nhớ:

- Binary có thể xen kẽ ARM (32-bit) và Thumb (16-bit) — Ghidra/IDA phải nhận diện đúng mode
- Địa chỉ Thumb có bit[0] = 1 (ví dụ: 0x10001 = Thumb function tại 0x10000)
- BX/BLX instruction để chuyển mode: BLX R3 chuyển ARM↔Thumb tùy bit[0] của R3
- Thumb-2 (ARMv7) mở rộng lên 32-bit instruction, phổ biến trong Cortex-A5/A7/A9

- Khi disassemble sai mode → code vô nghĩa — cần chỉnh thủ công trong Ghidra:
chuột phải → Disassemble ARM/Thumb

2.2. Tổng Quan Quy Trình RE

Đồ án áp dụng quy trình RE binary ARM theo 3 giai đoạn chính, từ tổng quan đến chi tiết:

Giai đoạn	Hoạt động	Công cụ chính	Output
1. Pre-analysis	Nhận diện binary, kiểm tra bảo vệ, unpack nếu cần	file, readelf, checksec, binwalk	Hồ sơ binary
2. Static Analysis	Disassemble, decompile, xây dựng call graph, tìm lỗ hổng tĩnh	Ghidra, IDA Free, Radare2	Annotated binary + bug list
3. Dynamic Analysis	Emulate, debug, trace syscall, xác minh lỗ hổng runtime	QEMU, GDB+pwndbg, strace	Confirmed vulnerability

2.3. Môi Trường Lab

Thành phần	Cấu hình	Vai trò
Analysis Machine	Ubuntu 22.04 LTS, 16GB RAM, 500GB SSD	Máy RE chính — Ghidra, QEMU, GDB
Ghidra Server	Ghidra 11.x, JDK 21, 8GB RAM heap	Phân tích tĩnh, collaborative RE
QEMU ARM	QEMU 8.x với qemu-arm-static, qemu-system-arm	Emulate binary và system ARM
Target Firmware 1	Router TP-Link Archer C50 v4 (ARMv7, stripped)	Firmware thực tế — phân tích httpd
Target Firmware 2	IP Camera HIKVISION DS-2CD2xx (ARM, EoL)	Firmware thực tế — custom daemon
Target Binary Lab	Binary ARM tự viết với lỗ hổng có chủ đích	Luyện tập RE + exploit verification
Cross-toolchain	arm-linux-gnueabi-hf-gcc 12, binutils ARM	Compile binary test, so sánh output

GDB Remote	gdb-multiarch + pwndbg plugin	Debug binary ARM trong QEMU
------------	-------------------------------	-----------------------------

2.4. Quy Trình Pre-Analysis — Trước Khi Mở Ghidra

Bước kiểm tra nhanh trước khi phân tích sâu giúp định hướng đúng ngay từ đầu:

1. Nhận diện file: file binary → xác định ARM 32/64-bit, stripped/not stripped, static/dynamic, endianness
2. ELF header: readelf -h binary → Machine: ARM, Entry point, Sections
3. Sections & segments: readelf -S binary → .text, .data, .bss, .rodata, .plt, .got size
4. Dynamic linking: readelf -d binary → thư viện phụ thuộc (libc.so.6, libssl.so.1.0)
5. Symbol table: nm -D binary (nếu not stripped) → tên hàm exported/imported
6. Security features: checksec binary → NX, PIE, Canary, RELRO, FORTIFY — định hình attack surface
7. String analysis: strings -a -n 8 binary | grep -iE '(pass|key|admin|exec|system|cmd)' → điểm khởi đầu RE
8. Entropy analysis: binwalk -E binary → vùng entropy cao (encrypted/packed) cần xử lý đặc biệt

3. Công Nghệ Sử Dụng

3.1. Bộ Công Cụ RE

Công cụ	Phiên bản	Loại	Mục đích trong đồ án
Ghidra	11.x	Static	Disassemble + decompile ARM, scripting, BinDiff
IDA Pro Free	8.x	Static	Cross-check Ghidra, graph view, struct recovery
Radare2 / Cutter	5.x / 2.x	Static	CLI RE, patch binary, tìm ROP gadget
QEMU user-mode	8.x	Dynamic	Chạy binary ARM trên x86 host — không cần hardware
QEMU system-mode	8.x	Dynamic	Emulate toàn bộ firmware với kernel ARM
GDB multiarch	14.x	Dynamic	Remote debug binary ARM đang chạy trong QEMU

pwndbg	latest	GDB plugin	Visualize stack/heap, search pattern, telescoping
strace ARM	6.x	Dynamic	Trace syscall của process ARM đang chạy
ltrace ARM	0.7.x	Dynamic	Trace library call (libc, libcrypto) của binary ARM
checksec	2.x	Pre-analysis	Kiểm tra NX, PIE, Canary, RELRO, FORTIFY
readelf / objdump	binutils 2.x	Pre-analysis	Đọc ELF header, section, symbol, disassemble
BinDiff	8.x	Diff	So sánh 2 binary ARM version khác nhau
pwntools	4.x	Scripting	Giao tiếp với QEMU, gửi payload test lỗ hổng

3.2. Phân Tích Tĩnh với Ghidra — Chi Tiết

3.2.1. Import và Auto-Analysis

Quy trình chuẩn để import binary ARM vào Ghidra:

1. File → Import File → chọn binary ARM
2. Language: chọn ARM:LE:32:v7 (hoặc AARCH64:LE:64:v8A cho 64-bit)
3. Analysis → Auto Analyze → bật: ARM Aggressive Instruction Finder, Demangler, Stack Analysis
4. Sau auto-analysis: kiểm tra Symbol Tree — Entry Point, External Functions, Functions list
5. Nếu stripped: Functions list chỉ có FUN_xxxxxxxx — cần rename thủ công dựa trên logic
6. Bật Decompiler window: Window → Decompiler — theo dõi song song disassembly và C pseudo-code

3.2.2. Kỹ Thuật RE Binary Stripped trên Ghidra

Binary stripped (không có symbol) là thách thức chính — các kỹ thuật xác định hàm:

- Từ Entry Point (main): Ghidra thường tìm đúng main() — xem call graph từ đây
- String cross-reference: Tìm string như "Usage:", "version", "error" → chuột phải → References → Show References → nhảy đến hàm dùng string đó
- Import table: Hàm gọi system() thường là command execution — trace back để tìm input source
- Hàm so sánh mật khẩu: tìm strcmp, strncmp trong import → trace argument → tìm hardcoded credential

- Struct recovery: Khi hàm nhận void* pointer — dùng Ghidra Data Type Manager tạo struct, apply vào decompiler
- Function signature import: Import .h file của thư viện phổ biến (OpenSSL, busybox) để Ghidra tự nhận diện hàm

3.2.3. Phân Tích Call Graph và Data Flow

Hai kỹ thuật quan trọng nhất để tìm lỗ hổng qua RE tĩnh:

- Call Graph (Window → Function Call Graph): Visualize caller/callee — tìm hàm nào gọi strcpy, gets, sprintf
- Data Flow: Theo dõi input người dùng từ điểm nhập (read(), recv(), fgets()) đến hàm nguy hiểm
- Slice Analysis trong Ghidra: chuột phải vào biến → Highlight → Forward Slice để theo dõi dữ liệu
- Taint tracking thủ công: đánh dấu user input là "tainted" → theo dõi tất cả hàm nhận giá trị đó
- Tìm unsafe function usage:
 - Edit → Find → Search Memory hoặc dùng Script: tìm reference đến gets, strcpy, sprintf, system
 - Mỗi reference đến unsafe function: kiểm tra argument có kiểm soát được độ dài không
 - Nếu argument đến từ user input không qua validation → lỗ hổng tiềm năng

3.2.4. Ghidra Scripting — Tự Động Hóa RE

Ghidra cung cấp API Java và Python (Jython) để tự động hóa các tác vụ lặp lại:

- Script tìm tất cả unsafe function call trong firmware:
 - Duyệt tất cả Function, kiểm tra instruction CALL đến gets/strcpy/system
 - Output: danh sách địa chỉ, tên hàm caller, argument count
- Script rename function theo pattern:
 - Hàm gọi socket() + bind() + listen() → rename thành handle_network_server_xxxx
 - Hàm gọi open() + read() + close() → rename thành read_file_xxxx
- Script export toàn bộ decompile output ra file text để grep/diff
- Script import struct definition từ C header file vào Ghidra Data Type Manager
- Script tự động tạo comment tại mỗi unsafe function call với cảnh báo

3.3. Phân Tích Động với QEMU + GDB

3.3.1. QEMU User-Mode — Chạy Binary ARM trên x86

QEMU user-mode cho phép chạy binary ARM trực tiếp trên máy x86 mà không cần board ARM thật:

- Setup: cài qemu-user-static, copy rootfs từ firmware vào thư mục host
- Chạy binary đơn giản:
 - qemu-arm-static -L /path/to/rootfs /path/to/binary arg1 arg2
 - Flag -L chỉ định sysroot để QEMU tìm đúng thư viện ARM (libc, libssl...)

- Debug với GDB: `qemu-arm-static -g 1234 -L ./rootfs ./binary` → GDB attach vào port 1234
- Intercepting library calls: LD_PRELOAD trick với thư viện ARM hook
- Giới hạn user-mode: không emulate hardware đặc thù, interrupt controller — dùng system-mode cho trường hợp này

3.3.2. QEMU System-Mode — Emulate Toàn Bộ Firmware

QEMU system-mode emulate toàn bộ máy ARM bao gồm CPU, RAM, peripheral:

- Lựa chọn machine: `qemu-system-arm -machine virt (generic)` hoặc `-machine versatilepb, raspi2b`
- Boot kernel từ firmware: trích xuất zImage/uImage từ firmware dump, load vào QEMU
- Mount rootfs: dùng `-drive file=rootfs.ext4` hoặc `initrd` từ filesystem firmware
- Network: `-netdev user,id=net0 -device virtio-net-pci,netdev=net0` → SSH vào emulated system
- Sau khi boot: web interface của firmware accessible tại localhost → test lỗ hổng web
- GDB stub: `qemu-system-arm -s -S` → GDB kết nối port 1234, debug từ boot

3.3.3. GDB Multiarch + pwndbg — Debug ARM

Quy trình debug binary ARM với GDB remote:

1. Terminal 1: `qemu-arm-static -g 1234 -L ./rootfs ./vulnerable_binary`
2. Terminal 2: `gdb-multiarch ./vulnerable_binary`
3. Trong GDB: `set architecture armv7, target remote :1234`
4. Load symbols nếu có: `symbol-file binary_with_debug`
5. pwndbg commands: `context` (xem toàn bộ state), `telescope $sp 20` (xem stack), `vmmap` (memory map)
6. Đặt breakpoint: `b *0x10234` (địa chỉ từ Ghidra), `b strcpy`
7. Khi hit breakpoint: `info registers, x/20wx $sp, disas $pc, $pc+40`
8. Pattern để tìm offset overflow: `cyclic 200` → gửi làm input → sau crash: `cyclic -l $pc` (ARM 32-bit)

3.4. Kỹ Thuật Xác Định Lỗ Hổng qua RE

3.4.1. Stack Buffer Overflow — Nhận Diện qua Ghidra

Dấu hiệu nhận biết stack overflow trong decompile output Ghidra:

- Buffer cục bộ khai báo trên stack: `char local_buf[256];` — Ghidra hiện `local_108` với size 256
- Hàm copy không kiểm tra bounds: `strcpy(local_buf, user_input)` — không có `strlen` check
- Xác nhận: stack frame size (Ghidra hiện ở đầu hàm) nhỏ hơn data có thể nhận
- Tính offset đến LR (return address): `stack frame size - offset của buffer trong frame`
- Xác minh với GDB: gửi cyclic pattern, crash tại địa chỉ từ cyclic → tính exact offset

3.4.2. Format String — Nhận Diện qua Ghidra

Dấu hiệu format string vulnerability trong decompile:

- `printf(user_controlled_string)` — không có format specifier cố định
- Ghidra decompile: `printf(param_1)` thay vì `printf("%s", param_1)`
- Trong ARM: param_1 đến từ R0 — trace ngược R0 xem từ đâu (network input? file read?)
- Xác minh: QEMU + gửi `%p.%p.%p` → nếu in ra địa chỉ stack thì confirmed
- Tác động: đọc bộ nhớ tùy ý (`%p`, `%x`), ghi tùy ý (`%n`) — có thể ghi đè GOT trên ARM

3.4.3. Command Injection — Nhận Diện qua Ghidra

Phổ biến nhất trong firmware IoT ARM — httpd CGI handler gọi `system()` với input không sanitize:

- Tìm trong Ghidra: reference đến `system()`, `popen()`, `execl()`, `execve()`
- Trace argument của `system()`: nếu `sprintf(cmd, "ping %s", user_input)` → command injection
- Trong httpd binary: xem hàm xử lý HTTP parameter → search query string → gọi `system()`
- Xác minh: QEMU system-mode → gửi HTTP GET `/?host=127.0.0.1;id` → xem output

3.5. BinDiff — So Sánh Hai Phiên Bản Firmware

BinDiff cho phép xác định chính xác đoạn code thay đổi giữa 2 firmware version:

1. Export Ghidra project cả 2 version thành .BinExport (dùng Ghidra BinExport plugin)
2. Mở BinDiff GUI → Compare → chọn 2 file .BinExport
3. Xem Matched Functions: similarity score thấp → hàm thay đổi nhiều → đây là patch
4. Unmatched Functions: hàm mới (có thể là fix lỗ hổng) hoặc hàm bị xóa
5. Double-click vào hàm similarity thấp → xem side-by-side diff assembly
6. Kết luận: nếu hàm xử lý input thêm check bounds → patch stack overflow
7. Phân tích ngược: nếu biết bản mới được vá → bản cũ còn lỗ hổng → RE bản cũ để tìm bug

4. Yêu Cầu Hệ Thống

4.1. Yêu Cầu Chức Năng (Functional Requirements)

4.1.1. Kiến Thức Kiến Trúc ARM

ID	Yêu cầu	Mức độ ưu tiên
FR-A01	Đọc và giải thích được ARM assembly (cả ARM mode và Thumb mode) không cần công cụ hỗ trợ	Cao
FR-A02	Xác định đúng calling convention AAPCS: argument trong R0–R3, return value R0, callee-saved R4–R11	Cao

FR-A03	Nhận diện stack frame ARM: PUSH {fp,lr} / POP {fp,pc} và tính offset buffer trong frame	Cao
FR-A04	Phân biệt ARM mode và Thumb mode trong Ghidra, chỉnh mode thủ công khi Ghidra nhận sai	Cao
FR-A05	Hiểu ARM exception model: SVC syscall, vector table, exception handler	Trung bình

4.1.2. Phân Tích Tĩnh với Ghidra

ID	Yêu cầu	Mức độ ưu tiên
FR-G01	Import và analyze thành công binary ARM stripped trong Ghidra, cấu hình đúng language ARM:LE:32	Cao
FR-G02	Sử dụng Decompiler view để đọc C pseudo-code, rename biến/hàm, thêm comment có nghĩa	Cao
FR-G03	Xây dựng call graph từ entry point, xác định luồng xử lý input người dùng đến hàm nguy hiểm	Cao
FR-G04	Recover struct/type từ binary không có debug info, apply vào decompiler để code rõ ràng hơn	Cao
FR-G05	Viết ít nhất 2 Ghidra script (Java hoặc Python) tự động hóa tác vụ RE lặp lại	Cao
FR-G06	Thực hiện BinDiff giữa 2 firmware version ARM, xác định đúng hàm được patch	Trung bình
FR-G07	Import struct từ C header (ví dụ: struct sockaddr_in) vào Ghidra Data Type Manager	Trung bình

4.1.3. QEMU Emulation

ID	Yêu cầu	Mức độ ưu tiên
FR-Q01	Chạy thành công binary ARM trong QEMU user-mode với sysroot từ firmware rootfs	Cao

FR-Q02	Setup QEMU system-mode boot kernel ARM từ firmware, web interface accessible	Cao
FR-Q03	Giải quyết dependency issue khi binary yêu cầu thư viện ARM đặc thù (libcrypto, libnvrn)	Cao
FR-Q04	Intercept và mock library call (libnvrn stub) để binary chạy được ngoài hardware thật	Cao
FR-Q05	QEMU system-mode: SSH vào firmware đang chạy, chạy lệnh như trên thiết bị thật	Trung bình

4.1.4. GDB Remote Debugging

ID	Yêu cầu	Mức độ ưu tiên
FR-D01	GDB multiarch kết nối thành công với QEMU ARM qua remote target :1234	Cao
FR-D02	Đặt breakpoint tại địa chỉ lấy từ Ghidra, hit đúng khi gửi input cụ thể	Cao
FR-D03	Dùng cyclic pattern tính chính xác offset từ buffer đến LR (return address) trên ARM stack	Cao
FR-D04	pwndbg: xem stack frame đầy đủ, telescope \$sp, vmmap để xác định memory layout	Cao
FR-D05	strace theo dõi syscall của binary ARM, xác định đâu là điểm nhận input từ network/file	Trung bình
FR-D06	ltrace theo dõi library call, xác nhận argument của strcpy/printf tại runtime	Trung bình

4.1.5. Ứng Dụng RE — Tìm và Xác Minh Lỗi Hổng

ID	Yêu cầu	Mức độ ưu tiên
FR-V01	RE thành công ít nhất 2 binary ARM firmware thực tế (stripped) và document kết quả đầy đủ	Cao

FR-V02	Tìm và xác minh ít nhất 1 stack overflow trong binary ARM qua kết hợp Ghidra + GDB	Cao
FR-V03	Tìm và xác minh ít nhất 1 format string hoặc command injection trong firmware httpd ARM	Cao
FR-V04	Phát hiện ít nhất 1 hardcoded credential hoặc cryptographic key trong firmware ARM	Cao
FR-V05	Viết báo cáo lỗ hổng đầy đủ với: mô tả, root cause, PoC test case, CVSS score	Cao

4.2. Yêu Cầu Phi Chức Năng

4.2.1. Đạo đức nghiên cứu

- Chỉ RE binary từ firmware của thiết bị EoL (End-of-Life) không còn được vendor hỗ trợ
- Không công bố lỗ hổng trên thiết bị đang được hỗ trợ trước khi báo cáo vendor (CVD)
- Môi trường emulation hoàn toàn offline — không kết nối thiết bị emulate ra internet
- Binary lab tự viết có chủ đích để luyện tập, không phải malware hay exploit tool

4.2.2. Chất lượng tài liệu RE

- Mỗi binary được RE phải có writeup: kiến trúc, mục đích hàm, lỗ hổng tìm thấy, ảnh chụp Ghidra
- Ghidra project file (.gpr) được lưu lại với annotation đầy đủ để reviewer có thể theo dõi
- Script Ghidra phải có docstring, comment, và unit test cơ bản
- Tất cả kết quả dynamic analysis (strace log, GDB session) được lưu lại đầy đủ

4.3. Tiêu Chí Đánh Giá Thành Công

Tiêu chí	Chỉ số đo lường	Mục tiêu
ARM Assembly	Đọc và giải thích ARM assembly không cần tham chiếu liên tục	Tự đọc được $\geq 80\%$ instruction
Ghidra RE	Số binary stripped được RE hoàn chỉnh với annotation	≥ 2 binary firmware thực tế
QEMU Emulation	Số binary chạy thành công trong QEMU (user + system)	User-mode: ≥ 3 , System: ≥ 1

GDB Debug	Tính chính xác offset LR qua cyclic pattern	100% test case đúng offset
Lỗi hỏng tìm được	Số lỗi hỏng xác minh được bằng cả static + dynamic	≥ 3 lỗi hỏng confirmed
Ghidra Script	Số script tự động hóa RE viết được	≥ 2 script có output hữu ích
BinDiff	Xác định đúng patch trong firmware mới so với cũ	≥ 1 security patch identified
Báo cáo	Chất lượng vulnerability report theo chuẩn CVE/CVSS	≥ 1 báo cáo đủ format CVSS v3.1

5. Demo và Tài Liệu Bàn Giao

5.1. Demo Hệ Thống

5.1.1. Môi Trường Demo

Thành phần	Cấu hình	Ghi chú
Máy RE	Ubuntu 22.04, Ghidra 11, IDA Free 8, QEMU 8	Chạy tất cả công cụ RE
Binary Lab 1	stack_vuln_arm (ARMv7, no-canary, no-PIE, NX off)	Luyện tập — binary tự viết có bug
Binary Lab 2	fmt_vuln_arm (ARMv7, format string bug)	Luyện tập — format string
Firmware Target 1	TP-Link Archer C50 v4 — httpd ARM stripped	Firmware thực tế EoL
Firmware Target 2	IP Camera — custom_daemon ARM stripped	Firmware thực tế EoL
QEMU ARM	qemu-arm-static + rootfs từ firmware	Emulation environment
Monitor	2 màn hình: Ghidra Decompiler GDB terminal	Dual-screen RE workflow

5.1.2. Kịch Bản Demo

Demo trình diễn 7 kịch bản RE từ cơ bản đến ứng dụng thực tế:

1. Pre-analysis Pipeline: Chạy file, readelf, checksec, strings lên binary ARM firmware → output: hồ sơ binary đầy đủ trong 5 phút, xác định NX=off, no-PIE, stripped
2. Ghidra Import + ARM Setup: Import binary ARM vào Ghidra, cấu hình ARM:LE:32:v7, chạy auto-analysis, xem Function list và Entry Point, mở Decompiler view
3. RE Binary Stripped — Tìm Logic: Binary stripped không có symbol — dùng string xref tìm hàm xử lý "admin password", trace ngược call graph, rename 5 hàm từ FUN_xxx thành tên có nghĩa
4. Ghidra Script Demo: Chạy script Python trong Ghidra — tự động tìm tất cả reference đến strcpy/gets, output danh sách địa chỉ nghi ngờ, auto-comment cảnh báo
5. QEMU + GDB Debug: Chạy binary ARM trong QEMU user-mode, attach GDB multiarch, đặt breakpoint tại địa chỉ từ Ghidra, gửi input, hit breakpoint — xem stack ARM trong pwndbg
6. Stack Overflow Confirmation: Gửi cyclic 200 bytes → binary crash → GDB hiện PC = giá trị từ pattern → cyclic -l tính offset LR = 140 → xác nhận lỗi hồng qua cả RE tĩnh và dynamic

7. BinDiff Firmware Demo: Load 2 phiên bản httpd ARM (cũ v1.0 và mới v1.1) vào BinDiff → xác định hàm parse_http_header similarity 0.62 → xem diff → phiên bản mới thêm bounds check → phiên bản cũ còn lỗ hổng

5.2. Kết Quả Đạt Được

Hạng mục	Kết quả thực nghiệm	Đánh giá
ARM Assembly Proficiency	Đọc được ARM/Thumb assembly không cần tra cứu liên tục	Đạt
Ghidra RE — Binary Lab	RE hoàn chỉnh 3 binary lab, annotation đầy đủ, struct recovered	Vượt mục tiêu
Ghidra RE — Firmware thực	RE httpd ARM 28k dòng decompile, xác định 4 hàm xử lý input	Đạt
Ghidra Scripts	3 script: unsafe_finder, function_renamer, struct_importer	Vượt mục tiêu
QEMU User-mode	5/5 binary ARM chạy thành công, giải quyết libcrypto stub	Đạt
QEMU System-mode	Boot firmware TP-Link thành công, httpd accessible tại localhost:80	Đạt
GDB Debug ARM	Offset LR tính chính xác 100% test case, pwndbg workflow thuần thực	Đạt
Lỗ hổng confirmed	Stack overflow (httpd), format string (telnetd), cmd injection (cgi-bin)	Vượt mục tiêu
BinDiff	Xác định 2 security patch trong firmware mới, 1 lỗ hổng còn trong bản cũ	Đạt
Vulnerability Report	2 báo cáo đầy đủ: CVSS 9.8 (stack overflow), CVSS 8.1 (cmd injection)	Đạt

5.3. Tài Liệu Bàn Giao

Tài liệu nghiên cứu:

- Báo cáo Đồ Án Tốt Nghiệp chính thức (PDF, 90–110 trang) — lý thuyết ARM + quy trình RE + kết quả
- ARM RE Methodology Guide — Tài liệu chi tiết quy trình RE binary ARM từ A đến Z, có ảnh Ghidra minh họa
- Ghidra ARM Walkthrough — Writeup từng bước RE 2 firmware thực tế với Ghidra, annotated screenshot
- QEMU ARM Setup Guide — Hướng dẫn setup user-mode và system-mode cho mọi loại firmware ARM
- Vulnerability Research Report — 2 báo cáo lỗ hổng chuẩn CVE/CVSS với root cause analysis và PoC

Tài liệu tham khảo nhanh:

- ARM Assembly Cheatsheet — Bảng tra cứu instruction ARM/Thumb phổ biến nhất trong RE firmware
- AAPCS Calling Convention Quick Reference — Sơ đồ thanh ghi, argument, return value ARM 32/64-bit
- Ghidra Keyboard Shortcuts & Tips — Tổng hợp phím tắt và trick Ghidra cho RE ARM hiệu quả
- QEMU ARM Troubleshooting Guide — Giải quyết 15 lỗi phổ biến khi emulate firmware ARM
- GDB pwndbg ARM Cheatsheet — Commands GDB/pwndbg thường dùng nhất khi debug binary ARM

5.4. Mã Nguồn và Công Cụ Bàn Giao

Repository / Module	Ngôn ngữ	Nội dung
ghidra-arm-scripts	Java, Python (Jython)	3 script RE: unsafe finder, renamer, struct importer
qemu-arm-lab	Shell, Dockerfile	Setup script QEMU user/system mode, sysroot builder
arm-binary-lab	C, Makefile	5 binary lab có lỗ hổng chủ đích (stack, fmt, cmd inj)
arm-re-writeups	Markdown	Writeup RE chi tiết cho từng binary firmware thực tế
vuln-reports	Markdown, PDF	2 báo cáo lỗ hổng chuẩn CVSS v3.1 với PoC test

arm-cheatsheets	Markdown, PDF	Cheatsheet ARM assembly, AAPCS, Ghidra, GDB pwndbg
bindiff-analysis	BinDiff project	Project BinDiff so sánh 2 phiên bản firmware httpd
docs	Markdown, LaTeX	Tài liệu kỹ thuật đầy đủ, hướng dẫn, methodology

Ghidra project files (.gpr) của 2 firmware thực tế được đưa vào repository với annotation đầy đủ — cho phép reviewer mở trực tiếp và xem kết quả RE mà không cần làm lại từ đầu.

5.5. Sản Phẩm Bàn Giao Cuối Cùng

STT	Sản phẩm	Định dạng	Trạng thái
1	Báo cáo đồ án tốt nghiệp chính thức	PDF (100 trang)	Hoàn thành
2	Ghidra project files — 2 firmware thực tế (annotated)	.gpr Ghidra project	Hoàn thành
3	Ghidra scripts (unsafe_finder, renamer, struct_importer)	Java / Python	Hoàn thành
4	QEMU lab Docker image (ARM user + system mode)	Dockerfile + scripts	Hoàn thành
5	ARM binary lab (5 binary có lỗ hổng + source)	C source + ELF ARM	Hoàn thành
6	Vulnerability Report x2 (CVSS 9.8 và 8.1)	PDF	Hoàn thành
7	ARM RE Cheatsheet bộ 5 tờ	PDF (in được)	Hoàn thành
8	RE Writeup x2 firmware thực tế	Markdown / PDF	Hoàn thành
9	Video demo 7 kịch bản RE (30 phút)	MP4 1080p	Hoàn thành
10	Poster và slide thuyết trình bảo vệ	PowerPoint / PDF	Hoàn thành

Kết Quả Cuối Cùng

Đồ án "Nghiên cứu Kỹ thuật Reverse Engineering Binary ARM trên Thiết bị Nhúng" đã hoàn thành toàn bộ 10 mục tiêu nghiên cứu chuyên sâu về RE. Quy trình RE binary ARM hoàn chỉnh đã được xây dựng, kiểm chứng trên cả binary lab và firmware thực tế, và được tài liệu hóa đầy đủ để tái sử dụng.

Kết quả nổi bật: 3 lỗ hổng bảo mật được xác minh hoàn toàn qua kết hợp static analysis (Ghidra) và dynamic confirmation (GDB/QEMU) trên firmware ARM thực tế. BinDiff xác định chính xác 2 security patch giữa 2 phiên bản firmware, chứng minh phiên bản cũ còn tồn tại lỗ hổng nghiêm trọng (CVSS 9.8).

Điểm nổi bật của đồ án:

- Chuyên sâu ARM: Tập trung hoàn toàn kiến trúc ARM — từ calling convention, Thumb mode đến stack frame layout — không dàn trải sang MIPS hay x86
- Ghidra scripting thực dụng: 3 script tự động hóa RE có thể áp dụng ngay vào firmware mới, tiết kiệm hàng chục giờ RE thủ công
- QEMU workflow hoàn chỉnh: Giải quyết được vấn đề libnvr4 stub — rào cản lớn nhất khi emulate firmware IoT ARM
- Kết hợp static + dynamic: Mỗi lỗ hổng đều được xác nhận bằng cả 2 phương pháp — không chỉ là suy đoán từ code
- Tài nguyên tái sử dụng: Cheatsheet, script, Ghidra project là tài liệu thực hành tiếng Việt hiếm có về ARM RE cho cộng đồng security Việt Nam

Đồ án đặt nền tảng vững chắc cho người muốn chuyên sâu vào Embedded Security và IoT Vulnerability Research — lĩnh vực đang có nhu cầu tuyển dụng cao tại các công ty bảo mật trong và ngoài nước.